[1]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

[2]:
```
kalimati_df=pd.read_csv("kalimati_historical_dataset.csv")
kalimati_df
```

[2]:

	Tomato Big(Nepali)	1/5/2021	Kg	50	60	55
0	Tomato Big(Indian)	1/5/2021	Kg	50	60	55
1	Tomato Small(Local)	1/5/2021	Kg	30	35	32.5
2	Tomato Small(Tunnel)	1/5/2021	Kg	30	35	32.5
3	Tomato Small(Indian)	1/5/2021	KG	40	45	42.5
4	Tomato Small(Terai)	1/5/2021	KG	40	45	42.5
...	...	...	...	...	...	...
96474	Tomato Small(Indian)	2023-09-28	KG	Rs 45.00	Rs 55.00	Rs 50.00
96475	Tomato Small(Local)	2023-09-28	KG	Rs 50.00	Rs 60.00	Rs 55.00
96476	Tomato Small(Tunnel)	2023-09-28	KG	Rs 60.00	Rs 75.00	Rs 69.00
96477	Turnip A	2023-09-28	KG	Rs 70.00	Rs 80.00	Rs 75.00
96478	Water Melon(Green)	2023-09-28	KG	Rs 50.00	Rs 60.00	Rs 56.67

96479 rows × 6 columns

[3]:
```
kalimati_df.columns=["Vegetables","Date","Weight","Minimum","Maximum","Average"]

kalimati_df
```

[4]:
```
kalimati_df.info()
```

```
<class 'pandas.core.frame.DataFrame'> •••
```

[5]:
```
kalimati_df["Date"]=pd.to_datetime(kalimati_df["Date"],yearfirst=True,format='mixed')
```

```
•••
```

[6]:
```
kalimati_df
```

```
<div> •••
```

[7]:
```
kalimati_df["Weight"]=kalimati_df["Weight"].str.lower()
kalimati_df
```

```
<div> •••
```

[8]:
```
for i in ["Minimum","Maximum","Average"]:
    kalimati_df[i]=kalimati_df[i].astype(str).str.replace("Rs","")
    kalimati_df[i]=pd.to_numeric(kalimati_df[i])
```

```
•••
```

[9]:
```
kalimati_df=kalimati_df[kalimati_df["Average"]>0]
kalimati_df
```

```
<div> •••
```

[10]:
```
kalimati_df.info()
```

```
<class 'pandas.core.frame.DataFrame'> •••
```

[11]:
```
kalimati_df["Vegetables"].nunique()
```

```
133 •••
```

:
```
highest_traded_vegetables=kalimati_df["Vegetables"].value_counts()
```

```
•••
```

:
```
top_traded_vegetables=highest_traded_vegetables.head(5).index.to_list()
fig,axes=plt.subplots(5,1,figsize=(14,18))
for i,vegies in enumerate(top_traded_vegetables):
    subset=kalimati_df[kalimati_df["Vegetables"]==vegies]
    axes[i].plot(subset["Date"],subset["Average"],color='green')
    axes[i].fill_between(subset["Date"],subset["Minimum"],subset["Maximum"],alpha=0.2,color="red")
    axes[i].set_title(vegies)
    axes[i].set_ylabel("NPR/KG")
plt.show()
```

```
<Figure size 1400x1800 with 5 Axes> •••
```

:
```
d=kalimati_df[kalimati_df["Vegetables"]=="Cauli Local"].copy()
d=d.sort_values("Date").reset_index(drop=True)
d
```

```
<div> •••
```

:
```
def make_features(kalimati_df,Vegetables):
    d=kalimati_df[kalimati_df["Vegetables"]==Vegetables].copy()
    d=d.sort_values("Date").reset_index(drop=True)
    d["lag_1"]=d["Average"].shift(1)
    d["lag_7"]=d["Average"].shift(7)
    d["lag_30"]=d["Average"].shift(30)

    d["roll_mean_7"]=d["Average"].rolling(7).mean()
    d["roll_std_7"]=d["Average"].rolling(7).std()
    d["roll_mean_30"]=d["Average"].rolling(30).mean()
    d["roll_std_30"]=d["Average"].rolling(30).std()

    d["month"]=d["Date"].dt.month
```

```
    d["month"]=d["Date"].dt.month
    d["day_of_week"]=d["Date"].dt.dayofweek
    d["day_of_year"]=d["Date"].dt.dayofyear

    d["is_dashain"]=d["month"].isin([9,10]).astype(int)
    d["is_monsoon"]=d["month"].isin([6,7,8]).astype(int)

    d["pct_change"]=d["Average"].pct_change(7)
    d=d.dropna()
    return d
```

•••

16]:
```
all_features=[]
for c in kalimati_df["Vegetables"].unique():
    try:
        all_features.append(make_features(kalimati_df,c))
    except:
        pass
master=pd.concat(all_features)
```

•••

17]:
```
master
```

5]:
```
def label_spikes(d):
    q1 = d["Average"].rolling(window=30, min_periods=1).quantile(0.25)
    q3 = d["Average"].rolling(window=30, min_periods=1).quantile(0.75)
    iqr = q3 - q1

    uthreshold = q3 + 1.5 * iqr
    lthreshold = q1 - 1.5 * iqr


    d["anomaly_label"] = 0
    d.loc[d["Average"] > uthreshold, "anomaly_label"] = 1
    d.loc[d["Average"] < lthreshold, "anomaly_label"] = -1

    return d
master = master.reset_index(drop=True)
master = master.groupby("Vegetables", group_keys=False).apply(label_spikes)
```

```
C:\Users\Loq\AppData\Local\Temp\ipykernel_19168\1576663303.py:16: DeprecationWarning: DataFrameGroupBy.apply operated on the grouping columns. This behav
```

9]:
```
master
```

 •••

[20]:
```python
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_absolute_percentage_error
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
import joblib

# 1. Define Features
FEATURES = [
    "lag_1",
    "lag_7",
    "lag_30",
    "roll_mean_7",
    "roll_std_7",
    "roll_mean_30",
    "roll_std_30",
    "month",
    "day_of_week",
    "day_of_year",
    "is_dashain",
    "is_monsoon",
    "pct_change",
    "anomaly_label"
]

le = LabelEncoder()
master["Vegetables_enc"] = le.fit_transform(master["Vegetables"])
FEATURES_FULL = FEATURES + ["Vegetables_enc"]

X = master[FEATURES_FULL]
y = master["Average"]

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, shuffle=False)
model = RandomForestRegressor(n_estimators=300, max_depth=10, random_state=42, n_jobs=-1)
model.fit(X_train, y_train)
preds = model.predict(X_test)
mape = mean_absolute_percentage_error(y_test, preds)
print(f"Test MAPE: {mape:.4f}")
```

```python
joblib.dump(model, "C:/Users/Loq/OneDrive/Desktop/Buisness-Data Analyst/Projects/Kalimati_project/models/random-forest.pkl")
joblib.dump(le, "C:/Users/Loq/OneDrive/Desktop/Buisness-Data Analyst/Projects/Kalimati_project/models/label_encoder.pkl")
```

```
Test MAPE: 0.0345
```

l]:
```python
d.iloc[-1]
```

```
Vegetables Cauli Local
```

2]:
```python
def forecast_7days_direct(Vegetables_name, kalimati_df, model, le):
    d = make_features(kalimati_df, Vegetables_name)
    d = label_spikes(d)
    last_row = d.iloc[-1].copy()
    last_row['Vegetables_enc'] = le.transform([Vegetables_name])[0]
    MODEL_FEATURES = [
        'lag_1', 'lag_7', 'lag_30',
        'roll_mean_7', 'roll_std_7', 'roll_mean_30', 'roll_std_30',
        'month', 'day_of_week', 'day_of_year',
        'is_dashain', 'is_monsoon', 'pct_change',
        'anomaly_label',
        'Vegetables_enc'
    ]
    forecasts=[]
    for day in range(1,8):
        row = pd.DataFrame([last_row[MODEL_FEATURES]])
        preds = model.predict(row)[0]
        forecasts.append(round(float(preds), 2))

        last_row['lag_30'] = last_row['lag_7']
        last_row['lag_7']  = last_row['lag_1']
        last_row['lag_1']  = preds

    return forecasts

preds = forecast_7days_direct('Tomato Big(Nepali)', kalimati_df, model, le)
print("7-day forecast (NPR/KG):", preds)
```